2022 29th Asia-Pacific Software Engineering Conference (APSEC) | 978-1-6654-5537-4/22/$31.00 ©2022 IEEE | DOI: 10.1109/APSEC57359.2022.00042

Optimal Refinement-based Array Constraint Solving for Symbolic Execution

Meixi Liu*†, Ziqi Shuai*†‡, Luyao Liu*, Kelin Ma*, Ke Ma*
* College of Computer, National University of Defense Technology, Changsha, China
† State Key Laboratory of High Performance Computing, National University of Defense Technology, Changsha, China
Email: {liumeixi, szq, lyliu, kelinma, ke}@nudt.edu.cn
‡ Corresponding author

***Abstract*—Array constraint solving is widely adopted by the existing symbolic execution engines for encoding programs precisely. The counterexample-guided abstraction refinement (CEGAR) based method is state-of-the-art for array constraint solving. However, we observed that the CEGAR-based method may need many refinements to solve the array constraints produced by the symbolic executor, which decreases the performance of constraint solving. Based on the observation, we propose a machine learning-based method to improve the efficiency of the CEGAR-based array constraint solving. Our method adaptively turns on or off the CEGAR loop according to different solving problems. We have implemented our method on the symbolic executor KLEE and its underlying CEGAR-based constraint solver STP. We have conducted an extensive experiment on 55 real-world programs. On average, our method increases the number of explored paths by 21%. The results of the extensive experiments on real-world C programs show the effectiveness of our method.**

***Index Terms*—symbolic execution, constraint solving, array SMT theory, machine learning**

I. Introduction

As a precise program analysis technique, symbolic execution [1] is useful for many software engineering activities: vulnerability detection [2], software verification [3] [4], program repair [5] and more. The main idea of symbolic execution is to take symbolic value as input, rather than concrete value. Based on the symbolic input, symbolic executors can collect symbolic expressions when executing program instructions. Once a branch instruction is reached, symbolic execution will fork to explore both branches and append the branch condition to the current *path constraint* (PC), *i.e.*, a first-order quantifier-free SMT formula over symbolic expressions. Thus the problem of path feasibility is transformed to the satisfiability problem of related path constraint, which can be determined by constraint solver. A satisfiable result from constraint solver indicates that the path is feasible, otherwise the exploration of the path is terminated. In this way, symbolic execution can systematically explore the path space of analyzed program. Theoretically, exhausted symbolic execution is both sound and complete [6].

It is straightforward to see that path constraint solving plays a crucial role in symbolic execution. Actually, recent advances in constraint solving techniques lead to the success of symbolic execution [7]. An efficient constraint solver greatly speeds up the path exploration and thereby enhances the scalability of symbolic execution. However, due to its inherent complexity, constraint solving is still a major bottleneck [6] [8]. Specifically, advanced SMT theories are necessary to describe complex data structures in programs, which result in complex path constraints. For instance, array is one of the most widely used data structures and is ubiquitous in programs, no matter the programming language. Usually, symbolic executors employ array SMT theory to encode the array operations in programs. Although array SMT theory simply provides two array terms for encoding, *i.e.*, Read and Write, the satisfiability problem for the theory is NP-Complete [9].

Because of its clear importance, array SMT theory is supported by lots of modern SMT solvers such as Z3 [10], STP [11] and Boolector [12]. These solvers mainly adopt two different methods to address the problem of array constraint solving. The first method, *a.k.a.* combinatory array logic (CAL) based method [13], uses a simple core of combinators and reduces it to the theory of uninterpreted functions. The other method is counterexample-guided abstraction refinement (CEGAR) based method [11], which iteratively constructs and refines abstraction until the array constraint is disproved or the solution is found. In detail, the CEGAR-based method abstracts the original array constraint through removing array terms, solves the abstracted constraint to obtain counterexamples and finally uses these counterexamples to refine the abstract constraint by introducing array axioms defined by array SMT theory. Besides, lemmas on demand [14] method is quite similar to the CEGAR-based method. Among the different methods, the CEGAR-based method is the state-of-the-art when solving path feasibility constraints in symbolic execution [15] [16].

Despite the high efficiency, the CEGAR-based array constraint solving method still has lots of challenges. We have observed that the CEGAR-based method may fail to terminate with limited iterations in many solving problems, which indicates that it is not always appropriate to apply the CEGAR-based method to some solving cases. In these cases, the extra overhead introduced by the CEGAR-based method, including the cost of solving abstracted constraint and counterexample generation, is overwhelming. Naturally, we have the following question: Can we recognize cases in which the CEGAR-based method can terminate fast enough to gain the performance benefit? It is obviously understandable

978-1-6654-5537-4/22/$31.00 ©2022 IEEE
DOI 10.1109/APSEC57359.2022.00042

that precisely recognizing these solving problems and only employing the CEGAR-based method for them will boost the constraint solving.

Based on the above observation, we proposed a machine learning-based intelligent selection method to improve the efficiency of the CEGAR-based array constraint solving in symbolic execution by adaptively turning on or off the method according to different solving problems. We resort to machine learning technique because it is well-known for classification problems, especially binary classification. Specifically, we use a dataset of formula generated from symbolic execution to train an offline machine learning model. Then, the model will be used for predicting whether the CEGAR loop is skipped.

We have implemented our approach on a state-of-the-art symbolic executor KLEE and its default underlying constraint solver STP. STP adopts a CEGAR-based method to solve array constraint. To fully evaluate our approach, we have conducted an extensive experiment on 55 real-world programs. The experimental results show that our approach can greatly boost the array constraints solving and then enhance the efficiency of symbolic execution. On average, our method increases the number of explored paths by 21% under DFS search strategy.

The remainder of this paper is organized as follows. Section II gives the background of the paper. Section III presents our framework and shows the details of the framework. Section IV gives the implementation and the evaluation. The related work is discussed and compared in Section V. Finally, the conclusion is drawn in Section VI.

II. Background

In this section, we will introduce the background of the paper, *i.e.*, the theory of array and the CEGAR-based array constraint solving method. Then, we will use an example to show the awkwardness of the CEGAR-based method.

A. The Theory of Arrays

Since memory can be regarded as a large array in a program, the theory of arrays is widely employed to reason about the behaviors of program. Given the context of symbolic execution, we only introduce the non-extensional theory of arrays. First of all, the theory of arrays [7] defines the following new sort symbol.

$$(Array\ Index\ Value) \tag{1}$$

The *Array* sort takes two sort parameters which are the sort of the index (*Index*) and the sort of the array elements (*Value*), respectively. Based on the new *Array* sort, the theory of arrays defines two operations, *Read* and *Write*, to load and store the contents of an array at a given position. The operations are defined as follow, where a is a variable of sort *Array*, i is a variable of sort *Index* and v is a variable of sort *Value*.

$$Read(a, i) \mid Write(a, i, v) \tag{2}$$

$Read(a, i)$ returns the value of array a at index i, while $Write(a, i, v)$ returns a new array that is the same as array a except that the value at index i in new array is updated to v. Using these two operations, the theory of arrays is able to model all array operations in programs.

In addition, the theory of arrays also provides the following axioms for array constraint solving, *i.e.*, the read axiom (3) and the read-over-write axiom (4).

$$\forall i, j.\ (i = j \Rightarrow Read(a, i) = Read(a, j)) \tag{3}$$

$$\begin{cases} \forall i, j.\ (i = j \Rightarrow Read(Write(a, j, v), i) = v) \\ \forall i, j.\ (i \neq j \Rightarrow Read(Write(a, j, v), i) = Read(a, i)) \end{cases} \tag{4}$$

The read axiom states that two read terms have the same value if their indices are equal. The read-over-write axiom states that loading value from an updated array at the modified position just gets the written value. Otherwise, it is the same as the loading value from the old array, *i.e.*, $Read(a, i)$. In general, the theory of arrays is often used together with other theories like the theory of bit-vectors to form a combined theory for encoding program behaviors. For example, symbolic execution usually uses SMT formulas in the QF_ABV logic to describe path constraints, which are quantifier-free formulas over the theory of bit-vectors and bit-vector arrays.

B. The CEGAR-based Array Constraint Solving Method

Counterexample-Guided Abstraction Refinement (CEGAR) is a classical paradigm and a standard framework in many research areas of computer science [17] [18] [19]. Similarly, the CEGAR-based method is introduced for array constraint solving in the seminal paper of STP [11]. Figure 1 gives the framework of the CEGAR-based method for solving SMT formulas in the QF_ABV logic.

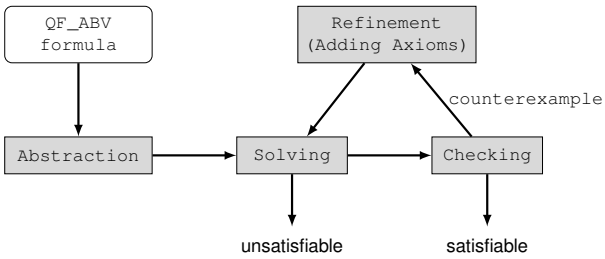


Fig. 1: The CEGAR-based array constraint solving framework.

The framework has two basic steps, abstraction and refinement. The abstraction is aimed to remove array terms in the original QF_ABV formula C. Firstly, it removes all write terms by applying the read-over-write axiom (Axiom (4)). Then, each read term in the write-free formula is replaced by a fresh bit-vector variable v. The mapping between the read term and v is recorded to generate read axioms in the following refinement. After abstraction, we get an abstract bit-vector constraint C_a without any array term. It is comprehensible that C_a is an over-approximation of C, which means C implies C_a ($C \Rightarrow C_a$). The iterative part of the CEGAR framework starts with C_a.

```
1  int main() {
2     int a[5] = {0, 1, 2, 3, 4};
3     int x = Symbolize();
4     if (a[x] > 10) {
5        assert(false);
6     }
7     return 0;
8  }
```

Fig. 2: An illustrative program

In the solving part, since C_a is a bit-vector constraint, the framework employs a bit-vector solver to decide it. A bit-vector solver often converts the input bit-vector constraint to an equisatisfiable SAT problem instance [7] and invokes SAT solver for solving. If the solving result is unsatisfiable, then C is unsatisfiable as well. Otherwise, the checking part will check whether the solution satisfies C. If so, then C is satisfiable and shares same solution with C_a. If not, the solution of C_a is a counterexample. The refinement part will use a counterexample-guided heuristic to select some read axioms (Axiom 3) for refining. An effective heuristic is to pick up the axioms that are violated by the counterexample [11]. These axioms are conjoined to C_a to refine the old abstract formula. Finally, the refined new C_a will be sent to the solving part to start a new iteration.

In such manner, the framework expects to disprove the original formula C or find the solution of C in a few iterations, simplifying the complexity of solved formula and reducing the solving cost of original QF_ABV formula. Because the number of read terms is finite, the solving is bound to terminate in limited iterations after all read axioms are conjoined to the abstract constraint C_a. In this case, the total refined C_a is equisatisfiable to C.

C. A Motivation Example

In this subsection, we use an illustrative example to motivate our work. The program in Figure 2 is a typical scenario of array usage which contains a buggy line, *i.e.*, line 5. Suppose that the integer variable x is symbolic and x is within the scope of array a ($0 \leq x \leq 4$). To trigger the buggy line, symbolic execution will generate the following array constraint C.

$$Read(a, x) > 10 \wedge 0 \leq x \leq 4$$

where a is $\{0, 1, 2, 3, 4\}$. Now, we replace the read term ($Read(a, x)$) in C with a bit-vector variable v and record the following five read axioms.

$$\Big(\bigwedge_{i \in \{0,1,2,3,4\}} x = i \Rightarrow v = i \Big)$$

In the CEGAR-based array constraint solving method, these axioms are added to the abstracted constraint C_a step by step. In the beginning, C_a is $v > 10 \wedge 0 \leq x \leq 4$. Obviously, it is satisfiable. Suppose that the solution of C_a is $(v = 11, x = 0)$, which does not satisfy the original constraint C. Therefore, the solution is a counterexample and violates the following axiom.

$$x = 0 \Rightarrow v = 0$$

Thereby, the violated axiom is added to C_a. So we get a new C_a, which is a refined constraint as follows.

$$v > 10 \wedge 0 \leq x \leq 4 \wedge (x = 0 \Rightarrow v = 0)$$

Then, C_a is solved again. We can get a new counterexample and use the counterexample to refine C_a again. In this example, the iteration terminates and disproves C until C_a is totally refined, *i.e.*, all of five axioms are added to the initial C_a. Hence, we need 5 times of refinement and invoke the underlying bit-vector constraint solver 6 times in total, which leads to a large overhead.

However, because C is unsatisfiable, the CEGAR framework iterates too many times in vain. It is more reasonable to add all read axioms once to the initial C_a after checking its satisfiability. In this way, we only need to invoke the bit-vector solver 2 times.

III. Intelligent Selection Method

In this section, we will describe our intelligent selection method in detail. First, we will introduce the framework of symbolic execution which is combined with our method. Then, we will present the employed solving procedure of array constraint. Next, we will explain the intelligent selection algorithm. Finally, we will discuss some details of the algorithm.

A. Symbolic Execution Framework

Algorithm 1 explains how to integrate our intelligent selection method into the existing symbolic execution framework that employs a worklist-style procedure. The algorithm requires a program P as input and performs symbolic execution on P. At the very beginning, there is only one initial state s_0 in the worklist (Line 1). The algorithm uses an integer variable N to record the number of solved queries yet and a boolean variable $switch$ to indicate whether to keep the CEGAR loop. Initially, N is 0 (Line 2) and $switch$ is $True$ (Line 3).

Algorithm 1: SE(P)

Require: A program P.

1: $worklist = \{s_0\}$
2: $N = 0$
3: $switch = True$
4: **while** $worklist \neq \emptyset$ **and** *not timeout* **do**
5: $\quad s = Extract(worklist)$
6: $\quad (C, s') = Execute(P, s)$
7: $\quad$ **if** $N\%T = 0$ **then**
8: $\quad\quad switch = ISCEGAR(\text{C})$
9: $\quad$ **end if**
10: $\quad res = Solve(C, switch)$
11: $\quad$ **if** $res = satisfiable$ **then**
12: $\quad\quad worklist = worklist \cup s'$
13: $\quad$ **end if**
14: $\quad N = N + 1$
15: **end while**

With the initial state s_0, the worklist-based algorithm starts running until the worklist is empty or the time budget is

reached (Line 4). During the running, the algorithm extracts a state from the worklist to perform the symbolic execution (Line 5), which can be viewed as a search to the state space of program. There are lots of search strategies in modern symbolic executors, such as depth-first search (DFS) and breadth-first search (BFS). In this paper, we suppose that the symbolic executor adopts the DFS search strategy. During the symbolic execution, the path constraint C is collected and new state s' is generated (Line 6). It's worth noting that lines 7 - 9 are our main contribution. Traditional algorithm invokes the underlying constraint solver directly to solve C (Line 10). If the result is satisfiable, s' is pushed into the worklist (Lines 11 - 13). Finally, the algorithm increments N (Line 14).

Obviously, the traditional algorithm enables CEGAR in constraint solving by default. Our main contribution aims to assign a boolean value dependent on C to $switch$. The value is predicted by our intelligent selection method (*i.e.*, $ISCEGAR$). Due to the prediction overhead, it is not reasonable to invoke $ISCEGAR$ for each path constraint C. Given that symbolic execution usually issues many constraints with similar structures, invoking $ISCEGAR$ intermittently is a good choice, which can balance the effectiveness and prediction overhead. We use a period factor T to control the invocation of our method. Whenever we have solved T queries, $ISCEGAR$ is invoked and a decision is made for a period of time in the future. In our experiment, we compare different values of T. The experimental results show that setting T to 200 is reasonable, which can ensure the effectiveness and reduce the prediction overhead at the same time.

B. Array Constraint Solving Method

Algorithm 2 gives the framework of our array constraint solving method. The input of the algorithm is an array constraint C and the $switch$ flag, while the output is the solving result, *i.e.*, satisfiable (SAT), unsatisfiable (UNSAT) or timeout (TIMEOUT).

The algorithm starts with producing the axioms of C (Line 1). This step gets an axiom set A containing all axioms of C. Then, the algorithm abstracts C by replacing all read terms with new free variables (Line 2). The abstracted constraint is C_a. C_a is a bit-vector constraint. After that, the refinement part of the algorithm begins, *i.e.*, the loop.

Inside the loop, the satisfiability of C_a is checked first (Line 4). C_a is transformed into an equi-satisfiable CNF formula and decided by the underlying SAT solver. There are some solving results. If the result is unsatisfiable, due to the implication relation ($C \Rightarrow C_a$), the original constraint C is unsatisfiable as well (Lines 5 - 6). However, if the result is satisfiable, the satisfiability of C should be checked further (Line 7). The algorithm uses the satisfying assignment sol to check C. If sol also satisfies C, then a solution has been found (Lines 8 - 9). Otherwise, the assignment is a counterexample. Then, the algorithm refines the abstracted constraint by conjoining the axioms. If the switch flag is turned off, the algorithm adds all axioms once to C_a (Lines 10 - 11). It's worth noting that lines 10 - 11 are our main contribution. On the other side, if the switch flag is on, the algorithm adds some axioms in priority using the counterexample (Lines 12 - 14). These axioms are removed from A (Line 15). Finally, if the result is unknown, which indicates that SAT solver cannot give a specific answer in given time budget, the algorithm returns timeout.

Algorithm 2: SOLVE($C, switch$)

Require: The array constraint C and the flag $switch$.
Ensure: SAT, UNSAT or TIMEOUT.

```
1:  A = RecordAxioms(C)
2:  C_a = Abstract(C)
3:  while true do
4:    (res, sol) = CheckSAT(C_a)
5:    if res = UNSAT then
6:      return UNSAT
7:    else if res = SAT then
8:      if sol ⊨ C then
9:        return SAT
10:     else if switch = False then
11:       C_a = C_a ∧ A
12:     else
13:       A_S = SelectAxioms(sol, A, C)
14:       C_a = C_a ∧ A_S
15:       A = A − A_S
16:     end if
17:   else
18:     return TIMEOUT
19:   end if
20: end while
```

The loop is guaranteed to terminate because the number of axioms is limited. On the one hand, the loop terminates after 2 times of refinement at most if we turn off the switch. When all axioms in A are added into C_a, we get an equi-satisfiable constraint with C without any array terms. Thus the satisfiability of C can be definitely decided. On the other hand, the loop adds some axioms at each refinement and terminates when A is empty.

C. Intelligent Selection

Algorithm 3 presents the details of our intelligent selection method of CEGAR. The input is an array constraint C and the output is the flag $switch$. In the beginning, the algorithm transforms C into a SMT formula, *i.e.*, f_{SMT} (Line 1). Then, the algorithm obtains the declared function variable and assertion content, convert each assertion into an abstract syntax tree for f_{SMT} (Line 2) according to lexical analysis and syntax analysis. The AST tree has many equal leaf nodes. These nodes are merged and a directed acyclic graph dag is produced (Line 3). Based on the dag, the algorithm extracts the feature (Line 4), which will be clarified in the following subsection. Finally, the feature is passed to the loaded model which is trained offline. The model gives the final prediction based on the input feature (Line 6).

Algorithm 3: ISCEGAR(C)

Require: The array constraint C.
Ensure: The flag $switch$.
1: $f_{SMT} = \text{TRANSFORM}(C)$
2: $tree = \text{AST}(f_{SMT})$
3: $dag = \text{MERGELEAF}(tree)$
4: $feature = \text{EXTRACT}(dag)$
5: $model = \text{LOADMODEL}()$
6: $switch = \text{PREDICT}(model, feature)$
7: **return** $switch$

D. Feature Extraction

Algorithm 4 shows the detail of feature extraction. We use the classical bag of words (BoW) model to perform the feature extraction. The input of the feature extraction algorithm is a dag, and the output is the bag of words model BoW. BoW can be understood as putting all words in a file into one bag (Line 1), regardless of the problems of morphology and word order, that is, each word is independent of each other. Map the words appearing in the constraint to the thesaurus. If they appear, add 1 to the corresponding position, otherwise it will be 0 (Lines 2 - 3). It first transforms the dag node type into a bag of words library, and then traverses all the dag nodes. BoW is widely used in document classification, and the frequency of word occurrence can be used as the feature of training classifier.

Algorithm 4: EXTRACT(dag)

Require: The directed acyclic graph of formula dag.
Ensure: The bag of words representation of formula BoW.
1: $AllNode = \text{NODES}(dag)$
2: **for** $node \in AllNode$ **do**
3: $\quad BoW[node] \leftarrow BoW[node] + 1$
4: **end for**
5: **return** BoW

For the following example,

$$(not(= (a_1\ or\ a_2))) \tag{5}$$

There are four kinds of node types, *i.e.*, not, $=$, $variable$ and or. The corresponding BoW representation is,

$$\{not : 1, =: 1, variable : 2, or : 1\} \tag{6}$$

which keys are node types and values are the number of nodes in different types.

E. Discussion

Model accuracy and benchmark sets are the main threats to our experimental results. This is mainly due to the limited benchmarks we use and the generalization of the machine learning model. The accuracy of the machine learning model can be further improved. For the former, although the number and type of benchmarks may be insufficient, Coreutils is a widely used benchmark to evaluate symbolic execution performance. The current experimental results show that the method is effective. However, we plan to evaluate our prototype on more benchmarks in the next step. In terms of model accuracy, we will also try other models and parameters to further improve the prediction accuracy.

IV. IMPLEMENTATION AND EVALUATION

A. Implementation

We implemented our method based on KLEE and use STP as the backend solver and bit-vector SMT theory for encoding the path constraints. KLEE's version is 2.3-pre and STP's version is 2.3.3. We implemented the AST translation and Bag of Word embedding based on jSMTLIB [20].

B. Research Questions

To evaluate our method, we need to answer the following two research questions:

- **RQ1**: among many machine learning models, which one is the most appropriate with regard to our scenario?
- **RQ2**: *efficiency*, how efficient is our method? Here the efficiency means that the symbolic execution can solve more queries and explore more paths under the same time budget.

C. Experimental Setup

Table I lists all of our benchmark programs. The programs are real-world open-source C programs. In total, we have 55 benchmark programs. Among them, 43 programs are selected from Coreutils, the standard benchmark for symbolic execution community. In total, Coreutils have 89 programs. However, many of them hardly use array data structure and the symbolic execution doesn't issue array constraint. So we filter the programs whose symbolic execution doesn't issue enough array constraints that reach the CEGAR loop, e.g., the ratio is no more than 10%. The other 12 programs are also common benchmarks used in different research areas [21] [22]. This means that our goal is to improve the analysis efficiency of some relatively complex programs.

TABLE I: The benchmark C programs.

Program	C SLOC	Brief Description
Coreutils	36069	43 GNU core utilities programs
apr	61201	Apache portable runtime project
bc	12511	GNU's numeric processing language
clan	2187	A polyhedral representation extractor
clex	1998	A C language lexer
libguile	1503	A GNU extension language library
libqpol	2431	A policy analysis support library
rats_perl	2006	Perl scanner in RATS tool
rats_php	2234	PHP scanner in RATS tool
rats_py	2021	Python scanner in RATS tool
rust	2610	A rust language lexer
m4	78214	Unix macro processor
make	28407	GNU building system
Total	233392	**55 open source C programs**

Setup for RQ1. We selected 5 well-known machine learning models as candidates, including Bayesian [23], KNN [24], MLP [25], RF (Random Forest) [26] and XGBoost [27].

To generate the training set, we collected 6269 formulas generated by 20% of the experimental programs. We can mark each constraint as 0 or 1 by checking whether it applies to a CEGAR-based solution. If the solution method based on CEGAR can be terminated in limited refinement, the constraint is marked as 1. Otherwise, it is marked as 0. Note that some constraints do not trigger the CEGAR loop. We randomly label it as 1 or 0. The ratio of label 0 to label 1 in the dataset is 1:1. In addition, the dataset contains some formulas that the switch cannot clearly determine whether it should turn on or turn off. In model training, the ratio of our training data to test data is 7:3.

Setup for RQ2. To alleviate the impact of nondeterminism in KLEE, all experiments were carried out under the DFS search strategy and without KLEE's query optimizations. We compare three configurations in detail: vanilla KLEE [1], using assertion encoding [2] and using intelligent selection method. We perform symbolic execution on each program for 30 minutes. All experiments were carried out on a 16-core server with 3.3GHz CPU and 64G memory. The operating system is Ubuntu 20.04. In order to reduce the experimental error, all experiments were carried out three times and the experimental results were average values.

D. Experimental Results

Answer to RQ1. We use four indicators to evaluate the models, including precision, recall, f1 score and prediction time. Table II shows the detailed results of the five models. The first column gives the name of five models. The second column lists label. False label means to turn off CEGAR, while true label means to turn on CEGAR. The last four columns represent four indicators. The precision column represents the proportion of true (false) samples among the predicted true (false) samples. The recall column represents the proportion of samples predicted to be true (false) in true (false) samples. The f1 column represents the harmonic average of precision and recall, and the calculation method is as follows. The time column represents the sum of training and prediction time. Among the five models, RF and XGBoost have higher accuracy and are relatively close, but the prediction time cost of XGBoost is significantly lower than RF. Obviously, XGBoost model performs best out of the five models under our scenario. So we finally train the intelligent selection model through XGBoost.

$$\mathbf{f1} = 2 * \frac{\mathbf{precision} * \mathbf{recall}}{\mathbf{precision} + \mathbf{recall}} \tag{7}$$

[1]The vanilla KLEE uses a flushing way to encode arrays.
[2]The KLEE's version uses assertion encoding [28] [29].

TABLE II: The results of five machine learning models.

model	label	precision	recall	f1	time(ms)
Bayesian	**false**	**0.60**	**0.47**	**0.53**	**44698**
	true	**0.55**	**0.67**	**0.61**	
KNN	**false**	**0.76**	**0.65**	**0.70**	**195101**
	true	**0.69**	**0.79**	**0.74**	
MLP	**false**	**0.30**	**0.72**	**0.42**	**5990**
	true	**0.88**	**0.55**	**0.68**	
RF	**false**	**0.80**	**0.65**	**0.72**	**817243**
	true	**0.70**	**0.83**	**0.76**	
XGBoost	**false**	**0.81**	**0.65**	**0.72**	**193056**
	true	**0.70**	**0.85**	**0.77**	

Answer to RQ1: *Considering both positive and negative samples, XGBoost performs best among the five candidate models, and its time cost is acceptable.*

Answer to RQ2. In principle, the number of solved queries and the number of explored paths directly reflect the efficiency of the symbolic executor. If a symbolic executor can solve more constraints or explore more paths under a time threshold, it is considered to be more effective. Therefore, we selected solved queries and explored paths as the measurement criteria of the experiment. In addition, since the refined constraint will be solved again, and the iteration will continue until a solution is found or disproving constraint. We also record the number of refined queries as auxiliary information.

Table III shows the main results. The first column shows our benchmark programs. **VANILLA** represents the vanilla KLEE. **ASSERTION** represents the KLEE version using assertion encoding introduced before. **MODEL** represents the KLEE version using our intelligent selection method. For each different configuration, we show three numbers. **#SQ** shows the solved queries, the total number of queries generated in half an hour. **#RQ** shows refined queries, the total number of arriving in CEGAR in half an hour. And **#EP** shows explored paths, the total number of explored paths explored in half an hour.

As shown in the table, for a total of 55 tasks, MODEL can find the most explored paths for 46 tasks, ASSERTION can find the most explored paths for 7 tasks, and there are 2 tasks MODEL perform as well as ASSERTION. In addition, for all the explored paths, the average of MODEL is 1.21 times of ASSERTION, the average of ASSERTION is 1.33 times of VANILLA, and the average of MODEL is 1.6 times of VANILLA. These results prove the effectiveness of MODEL.

For the efficiency results, as shown in the table, compared with VANILLA, MODEL achieves 60% (1%∼1018%) speedups on average to find explored paths. These results show that our method is effective to improve the efficiency of constraint solving.

In order to further evaluate the efficiency of our method, we show the speedup of solved queries, refined queries and explored paths respectively, as shown in the figure 3. The X-

TABLE III: Experimental Results (**#SQ**: the total number of solved queries, **#RQ**: the total number of refined queries, **#EP**: the total number of the explored paths).

Program	VANILLA			ASSERTION			MODEL		
	#SQ	#RQ	#EP	#SQ	#RQ	#EP	#SQ	#RQ	#EP
base64	57763	0	2421	75236	25487	3050	97558	31028	4023
chgrp	57442	0	2410	75972	25670	3081	96480	30791	3979
chmod	63833	0	1888	78651	20403	2318	92857	26925	2680
chown	124162	0	6523	137769	12011	7241	156429	15539	8438
cp	74935	0	3542	88509	16384	4179	109339	20016	5157
csplit	73953	0	4544	90119	11668	5262	108401	15989	6072
df	48638	0	3961	64008	14123	5066	81110	16860	6230
dircolors	55942	0	2367	70031	24381	2855	89972	29358	3704
du	68183	0	3108	90921	20572	4148	117371	25448	5361
expr	42041	29242	884	87762	56955	3424	103073	68538	3770
fmt	20162	0	2374	20037	6432	2364	23581	7550	2994
head	74561	0	15978	77936	7892	16932	95688	9653	20723
hostid	57276	0	2404	74526	25371	3016	97246	31002	4010
hostname	64561	0	2825	79004	23525	3337	100948	31687	4238
ginstall	70045	0	3282	87202	16227	4108	107241	20411	5047
kill	18449	0	3017	66468	12525	11013	75709	14046	12550
ln	71986	0	3359	90045	16906	4221	108428	20902	5076
logname	57633	0	2418	75792	25575	3073	98075	31147	4048
mkdir	107554	0	6020	128530	16122	7193	152977	21336	8469
mkfifo	65836	0	3206	91018	23901	4443	118340	29310	5785
mknod	45787	522	1914	96970	20018	3833	111794	22511	4408
mktemp	76457	30	3520	87723	13258	4051	102402	15170	4754
mv	69866	0	3098	85089	16054	3819	104285	20386	4675
nice	84714	0	2284	93036	12817	2507	111490	16248	3054
nohup	52655	0	2983	76525	24306	4337	95288	28117	5401
od	64035	0	3338	76585	22857	3781	95536	27609	4533
pr	27649	105	5953	41365	17854	9240	48521	20350	10899
printf	28338	0	1235	56897	19907	2404	68307	24287	2800
rm	119024	2868	6658	135228	16014	7576	167195	18211	9374
rmdir	61359	0	2843	84724	26051	3937	111146	31805	5173
split	74282	39	3328	91099	18056	4013	105756	24941	4638
stat	5399	0	264	5326	2641	263	5573	2858	271
stty	57986	22	2441	76958	25810	3139	98827	30954	4102
tac	60690	0	8413	81949	18033	11594	100119	26292	14395
tail	56315	0	27205	74908	17588	36355	96707	19816	47083
touch	73950	0	3475	103287	25695	4861	110111	27151	5181
tr	35356	65	6263	52035	6751	6319	65935	7499	6348
tty	47675	0	21806	56386	11866	25872	66568	20182	30549
uname	57837	0	2426	76571	25925	3109	97711	31037	4031
unexpand	47830	0	9774	78306	42319	16087	88455	49273	18188
unlink	64331	0	2817	79375	23813	3353	101147	31698	4247
wc	48949	0	17063	68656	42856	24016	77910	51020	27281
whoami	57646	0	2419	75877	25535	3078	97999	31098	4044
apr	258	0	16	646	620	38	858	827	64
bc	1223	0	16	1840	963	22	1832	955	22
clan	1229	0	34	2517	1651	71	2371	1526	70
clex	1288	0	36	2374	1517	58	2358	1484	56
libguile	1696	0	40	2331	2104	66	2526	2292	74
libqpol	781	0	28	1584	830	40	1588	837	40
rats_perl	816	0	43	13477	11621	458	11666	10011	399
rats_php	1297	0	47	18737	16571	320	16720	14691	295
rats_python	805	0	40	17750	15444	546	14141	12312	447
rust	1047	0	30	2319	1345	58	2124	1130	43
m4	4073	81	272	8269	5656	878	8302	5691	876
make	12396	0	71	21802	16890	84	21146	15936	88

axis shows the benchmark programs ordered by the values of Y-axis. The Y-axis shows the relative increasing of the solved queries, refined queries and explored paths, which is defined as follows, where $N_{\mathbf{OPT}}$ denotes the number of explorations after using the optimization method, and $N_{\mathbf{ORI}}$ represents the number of original symbolic execution method.

$$\frac{N_{\mathbf{OPT}} - N_{\mathbf{ORI}}}{N_{\mathbf{ORI}}} \tag{8}$$

Figure 3a, 3b and 3c show the explored paths speedup, 33% (0%∼1265%) for ASSERTION - VANILLA, 21% (-26%∼68%) for MODEL - ASSERTION, 60% (1%∼1018%) for MODEL - VANILLA.

Figure 3d, 3e and 3f show the solved queries speedup, 34% (-1%∼2105%) for ASSERTION - VANILLA, 21% (-20%∼33%) for MODEL - ASSERTION, 62% (3%∼1657%) for MODEL - VANILLA.

From the results of the figure 3, the results of ASSERTION and MODEL are significantly better than those of the original KLEE. The results of MODEL are improved on the basis of ASSERTION, but the results of MODEL are slightly worse than those of ASSERTION in a few programs, which is due to the randomness of symbol execution and the accuracy of model prediction.

Hence, we can have the following result of our method's efficiency.

Answer to RQ2: *On average, our approach can increase the number of explored paths and solved queries by 21% under DFS search strategy, respectively.*

E. Threats to Validity

The internal threat to the validity of our work is brought by the nondeterminism in KLEE. KLEE is nondeterministic because of many factors including the search strategy and memory allocation. To alleviate the impact of nondeterminism in KLEE, we use DFS search strategy, a deterministic search strategy. In the meantime, all of our experiments are carried out three times, and the results are the average values. The external threats are the major threats. The training set of our model may be biased, since it is the set of formulas collected in symbolic execution of Coreutils programs. We will build a more extensive training set in the future work. Besides, the precision and recall of our model are not high enough. We plan to apply our method to more machine learning models.

V. Related Works

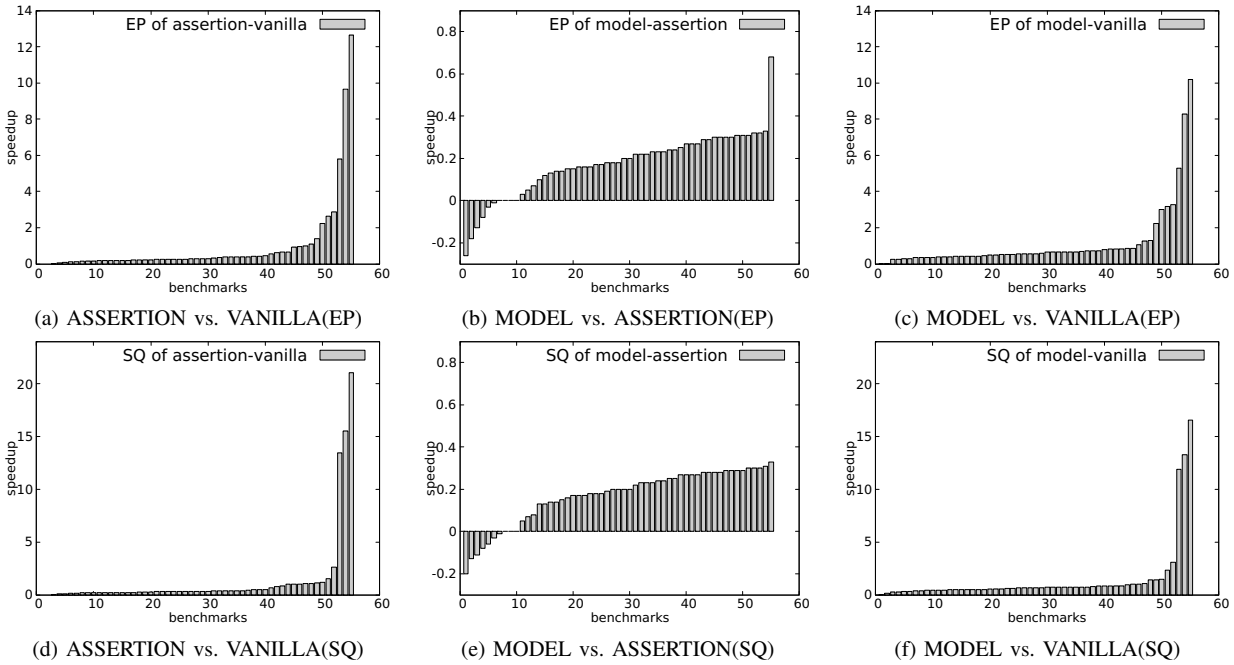
Constraint solving is the major bottleneck of symbolic execution technique. To boost the efficiency of constraint solving, many optimization techniques have been proposed. Our approach is closely related to these techniques. In this section, we will illustrate these techniques in two aspects.

A. Constraint Solving Optimizations in Symbolic Execution

Usually, constraint solving techniques in the context of symbolic execution can be roughly classified into two classes. The first class tries to relieve the burden of constraint solvers by reducing the size and complexity of constraints. Expression rewriting [30] [31] is a natural idea, which uses simplifications such as constant folding (*e.g.*, $3 + 2 \rightarrow 5$) and strength reduction (*e.g.*, $2 * x \rightarrow x + x$) to rewrite the expression into a simpler version. Similarly, KLEE-Array proposes two novel transformations, *i.e.,* index-based transformation and value-based transformation, to rewrite array constraint into an array term-free constraint, thereby avoiding the high cost of array constraint solving. Concretization is another efficient constraint reduction technique, which is widely employed by dynamic symbolic execution to simplify complex constraints using concrete values [32]. Mixed concrete-symbolic solving also tries to extend the concretization method to classical symbolic execution [33]. The second class reuses the previous solving results to prevent redundant constraint solving. The counterexample cache can exploit the relation between constraint sets, *e.g.*, a constraint set must be unsatisfiable if one of its subsets has been proven to be unsatisfiable [30]. The Green framework further provides a mechanism for reusing constraint solution across different programs and analysis [34]. As an extension of the Green framework, GreenTrie supports constraint reuse based on the logical implication relations among constraints [35]. However, the traditional works consider the constraint solver as a black box, leaving much room for improvement. Multiplex symbolic execution (MuSE) is the first work to use the constraint solver in a white-box manner [36]. Since constraint solving is a search process in principle, the final solution is usually found after many unsuccessful searches. MuSE uses partial solutions produced by these unsuccessful searches to construct program inputs, exploring multiple paths by one time of solving.

B. Machine Learning Techniques in Constraint Solving

Machine learning techniques have been hot topics in recent years. Many research areas including constraint solving can harness the power of machine learning to develop useful tools. Portfolio-based constraint solving method is a typical example [37] [38]. Since the developers of constraint solver usually design different heuristics for different classes of solving problems, existing constraint solvers are highly tuned for specific classes of solving problems. The method proposes to use machine learning techniques to train a classifier and then selects proper constraint solver which is suitable to solve the input constraint through the classifier. CSP-cNN [39] applies a convolutional neural network (CNN) model to predict the satisfiabilities of random Boolean binary CSPs with high prediction accuracies. Unlike CSP-cNN, DeepSolver [40] utilizes the existing constraint solutions in symbolic execution to train a deep neural network (DNN) model. Next, the DNN model is used to classify path constraints for their satisfiability during symbolic execution without invoking the expensive constraint solver. MLB [41] designs a novel method



(a) ASSERTION vs. VANILLA(EP) (b) MODEL vs. ASSERTION(EP) (c) MODEL vs. VANILLA(EP)

(d) ASSERTION vs. VANILLA(SQ) (e) MODEL vs. ASSERTION(SQ) (f) MODEL vs. VANILLA(SQ)

Fig. 3: Speedup of explored paths and solved queries

to transform the constraint solving problems in symbolic execution into optimization problems which can be solved by a sampling-validation style machine learning based optimization solver. As for SMT solving, FastSMT [42] presents a new method based on a combination of learning and synthesis techniques to synthesize a better solving strategy for SMT formulas.

VI. Conclusion

This paper presents a method to intelligently select the appropriate CEGAR switch for a given logic formula. This method can solve the constraint problem more effectively than using CEGAR algorithm for all formulas.

Our method uses the machine learning model of offline training to predict the appropriate CEGAR switch according to the given logic formula. We integrate our selection algorithm into KLEE and STP based symbolic execution frameworks, which are the most advanced C program symbolic execution engine and its default underlying constraint solver. The experimental results show that the method can effectively improve the efficiency of symbolic execution based on 55 C programs in the real world extensive evaluation benchmarks. On average, our method increases the number of explored paths by 21% compared to the baseline ASSERTION.

Next, we try to mine some patterns that are suitable or not suitable for CEGAR, and design heuristic rules to make up for the lack of model prediction. Besides, we will continue to look for better feature extraction methods, seek higher model prediction accuracy, and test on more real-world benchmarks.

VII. Acknowledgements

This research was supported by the NSFC Programs (No. 62172429 and 62032024).

References

[1] J. C. King, “Symbolic Execution and Program Testing,” *Commun. ACM*, vol. 19, no. 7, p. 385–394, jul 1976. [Online]. Available: https://doi.org/10.1145/360248.360252

[2] P. Godefroid, M. Y. Levin, and D. Molnar, “SAGE: Whitebox Fuzzing for Security Testing,” *Commun. ACM*, vol. 55, no. 3, p. 40–44, mar 2012. [Online]. Available: https://doi.org/10.1145/2093548.2093564

[3] J. Jaffar, V. Murali, J. A. Navas, and A. E. Santosa, “TRACER: A Symbolic Execution Tool for Verification,” P. Madhusudan and S. A. Seshia, Eds. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-642-31424-7_61

[4] H. Yu, Z. Chen, X. Fu, J. Wang, Z. Su, J. Sun, C. Huang, and W. Dong, “Symbolic Verification of Message Passing Interface Programs,” in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, ser. ICSE ’20. Association for Computing Machinery, 2020, p. 1248–1260. [Online]. Available: https://doi.org/10.1145/3377811.3380419

[5] S. Mechtaev, J. Yi, and A. Roychoudhury, “Angelix: Scalable Multiline Program Patch Synthesis via Symbolic Analysis,” in *Proceedings of the 38th International Conference on Software Engineering*, ser. ICSE ’16. Association for Computing Machinery, 2016, p. 691–701. [Online]. Available: https://doi.org/10.1145/2884781.2884807

[6] R. Baldoni, E. Coppa, D. C. D’elia, C. Demetrescu, and I. Finocchi, “A Survey of Symbolic Execution Techniques,” *ACM Comput. Surv.*, vol. 51, no. 3, may 2018. [Online]. Available: https://doi.org/10.1145/3182657

[7] D. Kroening and O. Strichman, *Decision Procedures - An Algorithmic Point of View, Second Edition*, ser. Texts in Theoretical Computer Science. An EATCS Series. Springer Berlin, Heidelberg, 2016. [Online]. Available: https://link.springer.com/book/10.1007/978-3-662-50497-0

[8] L. de Moura and G. O. Passmore, *The Strategy Challenge in SMT Solving*. Springer Berlin Heidelberg, 2013, pp. 15–44. [Online]. Available: https://doi.org/10.1007/978-3-642-36675-8_2

[9] P. J. Downey and R. Sethi, "Assignment Commands with Array References," *J. ACM*, vol. 25, no. 4, p. 652–666, oct 1978. [Online]. Available: https://doi.org/10.1145/322092.322104

[10] L. de Moura and N. Bjørner, "Z3: An Efficient SMT Solver," in *Tools and Algorithms for the Construction and Analysis of Systems*, C. R. Ramakrishnan and J. Rehof, Eds. Springer Berlin Heidelberg, 2008, pp. 337–340. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-540-78800-3_24

[11] V. Ganesh and D. L. Dill, "A Decision Procedure for Bit-Vectors and Arrays," in *Computer Aided Verification*, W. Damm and H. Hermanns, Eds. Springer Berlin Heidelberg, 2007, pp. 519–531. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-540-73368-3_52

[12] R. Brummayer and A. Biere, "Boolector: An Efficient SMT Solver for Bit-Vectors and Arrays," in *Tools and Algorithms for the Construction and Analysis of Systems*, S. Kowalewski and A. Philippou, Eds. Springer Berlin Heidelberg, 2009, pp. 174–177. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-642-00768-2_16

[13] L. de Moura and N. Bjørner, "Generalized, efficient array decision procedures," in *2009 Formal Methods in Computer-Aided Design*, 2009, pp. 45–52.

[14] R. Brummayer and A. Biere, "Lemmas on Demand for the Extensional Theory of Arrays," *Journal on Satisfiability, Boolean Modeling and Computation*, vol. 6, pp. 165–201, 05 2009.

[15] A. Sharma, "An Empirical Study of Path Feasibility Queries," *CoRR*, vol. abs/1302.4798, 2013. [Online]. Available: http://arxiv.org/abs/1302.4798

[16] H. Palikareva and C. Cadar, "Multi-solver Support in Symbolic Execution," in *Computer Aided Verification*, N. Sharygina and H. Veith, Eds. Springer Berlin Heidelberg, 2013, pp. 53–68. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-642-39799-8_3

[17] E. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith, "Counterexample-Guided Abstraction Refinement for Symbolic Model Checking," *J. ACM*, vol. 50, no. 5, p. 752–794, sep 2003. [Online]. Available: https://doi.org/10.1145/876638.876643

[18] S. Löwe, "Effective Approaches to Abstraction Refinement for Automatic Software Verification," Ph.D. dissertation, University of Passau, Germany, 2017. [Online]. Available: https://opus4.kobv.de/opus4-uni-passau/frontdoor/index/index/docId/481

[19] Á. Hajdu and M. Zoltán, "Efficient Strategies for CEGAR-Based Model Checking," *J. Autom. Reason.*, vol. 64, no. 6, pp. 1051–1091, 2020. [Online]. Available: https://doi.org/10.1007/s10817-019-09535-x

[20] D. R. Cok, "jSMTLIB: Tutorial, Validation and Adapter Tools for SMT-LIBv2," in *NASA Formal Methods*, M. Bobaru, K. Havelund, G. J. Holzmann, and R. Joshi, Eds. Springer Berlin Heidelberg, 2011, pp. 480–486. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-642-20398-5_36

[21] Z. Shuai, Z. Chen, Y. Zhang, J. Sun, and J. Wang, "Type and Interval Aware Array Constraint Solving for Symbolic Execution," in *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2021. Association for Computing Machinery, 2021, p. 361–373. [Online]. Available: https://doi.org/10.1145/3460319.3464826

[22] T. Kapus and C. Cadar, "A Segmented Memory Model for Symbolic Execution," in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2019. Association for Computing Machinery, 2019, p. 774–784. [Online]. Available: https://doi.org/10.1145/3338906.3338936

[23] A. McCallum and K. Nigam, "A comparison of event models for naive bayes text classification," in *AAAI 1998*, 1998.

[24] O. Kramer, *K-Nearest Neighbors*. Springer Berlin Heidelberg, 2013, pp. 13–23. [Online]. Available: https://doi.org/10.1007/978-3-642-38652-7_2

[25] D. W. Ruck, S. K. Rogers, and M. Kabrisky, "Feature Selection Using a Multilayer Perceptron," *Neural Network Comput*, vol. 2, pp. 40–48, 1990.

[26] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, pp. 5–32, 2001. [Online]. Available: https://doi.org/10.1023/A:1010933404324

[27] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16. Association for Computing Machinery, 2016, p. 785–794. [Online]. Available: https://doi.org/10.1145/2939672.2939785

[28] "Klee's github issue," 2021, https://github.com/klee/klee/issues/836 Accessed July 26, 2021.

[29] "Klee's github issue," 2021, https://github.com/klee/klee/pull/837 Accessed July 26, 2021.

[30] C. Cadar, D. Dunbar, and D. Engler, "KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs," in *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI'08. USENIX Association, 2008, p. 209–224. [Online]. Available: https://dl.acm.org/doi/10.5555/1855741.1855756

[31] D. A. Ramos and D. Engler, "Under-Constrained Symbolic Execution: Correctness Checking for Real Code," in *Proceedings of the 24th USENIX Conference on Security Symposium*, ser. SEC'15. USENIX Association, 2015, p. 49–64. [Online]. Available: https://dl.acm.org/doi/10.5555/2831143.2831147

[32] P. Godefroid, N. Klarlund, and K. Sen, "DART: Directed Automated Random Testing," in *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '05. Association for Computing Machinery, 2005, p. 213–223. [Online]. Available: https://doi.org/10.1145/1065010.1065036

[33] C. S. Păsăreanu, N. Rungta, and W. Visser, "Symbolic Execution with Mixed Concrete-Symbolic Solving," in *Proceedings of the 2011 International Symposium on Software Testing and Analysis*, ser. ISSTA '11. Association for Computing Machinery, 2011, p. 34–44. [Online]. Available: https://doi.org/10.1145/2001420.2001425

[34] W. Visser, J. Geldenhuys, and M. B. Dwyer, "Green: Reducing, Reusing and Recycling Constraints in Program Analysis," in *Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering*, ser. FSE '12. Association for Computing Machinery, 2012. [Online]. Available: https://doi.org/10.1145/2393596.2393665

[35] X. Jia, C. Ghezzi, and S. Ying, "Enhancing Reuse of Constraint Solutions to Improve Symbolic Execution," in *Proceedings of the 2015 International Symposium on Software Testing and Analysis*, ser. ISSTA 2015. Association for Computing Machinery, 2015, p. 177–187. [Online]. Available: https://doi.org/10.1145/2771783.2771806

[36] Y. Zhang, Z. Chen, Z. Shuai, T. Zhang, K. Li, and J. Wang, "Multiplex Symbolic Execution: Exploring Multiple Paths by Solving Once," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '20. Association for Computing Machinery, 2020, p. 846–857. [Online]. Available: https://doi.org/10.1145/3324884.3416645

[37] L. Xu, F. Hutter, H. H. Hoos, and K. Leyton-Brown, "SATzilla: Portfolio-Based Algorithm Selection for SAT," *J. Artif. Int. Res.*, vol. 32, no. 1, p. 565–606, jun 2008. [Online]. Available: https://dl.acm.org/doi/10.5555/1622673.1622687

[38] J. Scott, A. Niemetz, M. Preiner, S. Nejati, and V. Ganesh, "MachSMT: A Machine Learning-based Algorithm Selector for SMT Solvers," in *Tools and Algorithms for the Construction and Analysis of Systems*, J. F. Groote and K. G. Larsen, Eds. Springer International Publishing, 2021, pp. 303–325. [Online]. Available: https://doi.org/10.1007/978-3-030-72013-1_16

[39] H. Xu, S. Koenig, and T. K. S. Kumar, "Towards Effective Deep Learning for Constraint Satisfaction Problems," in *Principles and Practice of Constraint Programming*, J. Hooker, Ed. Springer International Publishing, 2018, pp. 588–597. [Online]. Available: https://doi.org/10.1007/978-3-319-98334-9_38

[40] J. Wen, M. Khan, M. Che, Y. Yan, and G. Yang, "Constraint Solving with Deep Learning for Symbolic Execution," *CoRR*, vol. abs/2003.08350, 2020. [Online]. Available: https://arxiv.org/abs/2003.08350

[41] X. Li, Y. Liang, H. Qian, Y.-Q. Hu, L. Bu, Y. Yu, X. Chen, and X. Li, "Symbolic Execution of Complex Program Driven by Machine Learning Based Constraint Solving," in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE 2016. Association for Computing Machinery, 2016, p. 554–559. [Online]. Available: https://doi.org/10.1145/2970276.2970364

[42] M. Balunović, P. Bielik, and M. Vechev, "Learning to Solve SMT Formulas," in *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, ser. NIPS'18. Curran Associates Inc., 2018, p. 10338–10349. [Online]. Available: https://dl.acm.org/doi/10.5555/3327546.3327694